| **Exp No: 8** | **Variational Autoencoder (VAE) for Learning a Latent Representation and Generating New Images** |
|---|---|

## AIM:

To implement a variational autoencoder (VAE) to generate new images from a given dataset, train it to learn the latent representation of the images, and generate new samples from the learned distribution.

## ALGORITHM / PROCEDURE:

1. Load the image dataset (1797 handwritten digit images of 8x8 grayscale pixels), scale the pixels to 0-1 and split the data 80% / 20% into training and test sets.
2. **Encoder:** a dense layer (64 to 64, ReLU) followed by two outputs, the mean `mu` and the log-variance of a 2-dimensional latent vector z.
3. **Reparameterization trick:** sample `z = mu + sigma * eps` with `eps ~ N(0, I)` so that gradients can flow through the sampling step.
4. **Decoder:** a dense layer (2 to 64, ReLU) and an output layer of 64 sigmoid units that gives the reconstructed image.
5. **Loss** (negative ELBO) = reconstruction loss (binary cross-entropy) + KL divergence between the encoder distribution and N(0, I).
6. Train with backpropagation and Adam (lr = 0.002, batch size 32, 300 epochs). All gradients were verified against numerical gradients.
7. Evaluate on the test images (reconstruction error and negative ELBO).
8. **Generate new images** by sampling z from N(0, I) and passing it through the decoder. Plot the latent space and the images decoded over a grid of z values.

**Fig 1: Structure of the VAE (training)**

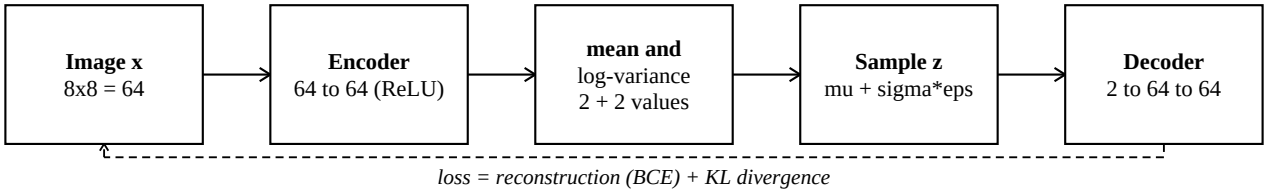
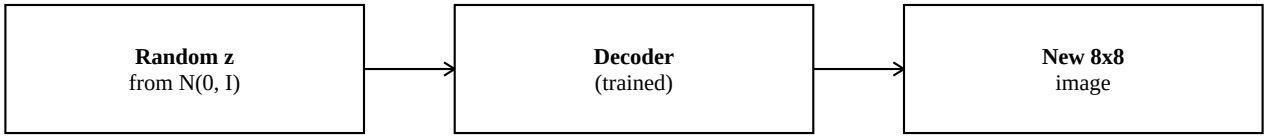



**Fig 2: Generating new images**



## Commands / Settings used:

| Part | Output size | Details |
|---|---|---|
| Input image | 64 | 8x8 pixels scaled to 0-1 |
| Encoder hidden layer | 64 | Dense + ReLU |
| Latent vector z | 2 | mean and log-variance, then sampling |
| Decoder hidden layer | 64 | Dense + ReLU |
| Output image | 64 | Dense + sigmoid |
| Loss / optimizer | | BCE + KL / Adam, lr 0.002, batch 32, 300 epochs |

## PROGRAM:

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(0)

# ---------- Data: 8x8 grayscale handwritten digit images (64 pixels, scaled 0-1) ----------
d = load_digits()
X, y = d.data / 16.0, d.target
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# ---------- VAE: 64 -> 64 -> (mu, log-variance) of 2 latent units -> 64 -> 64 ----------
D, H, Z = 64, 64, 2


def init(nin, nout, scale=1.0):
    return rng.standard_normal((nin, nout)) * np.sqrt(2 / nin) * scale, np.zeros(nout)


P = {}
P["W1"], P["b1"] = init(D, H)            # encoder hidden layer
P["Wmu"], P["bmu"] = init(H, Z, 0.1)     # mean of q(z|x)
P["Wlv"], P["blv"] = init(H, Z, 0.1)     # log-variance of q(z|x)
P["W3"], P["b3"] = init(Z, H)            # decoder hidden layer
P["W4"], P["b4"] = init(H, D)            # decoder output layer


def sigmoid(a):
    return 1 / (1 + np.exp(-a))


def encode(x):
    h = np.maximum(0, x @ P["W1"] + P["b1"])
    return h, h @ P["Wmu"] + P["bmu"], h @ P["Wlv"] + P["blv"]


def decode(z):
    g = np.maximum(0, z @ P["W3"] + P["b3"])
    return g, g @ P["W4"] + P["b4"]


def losses(x, logits, mu, lv):
    recon = np.sum(np.logaddexp(0, logits) - x * logits, axis=1)   # binary cross-entropy
    kl = -0.5 * np.sum(1 + lv - mu**2 - np.exp(lv), axis=1)         # KL(q(z|x) || N(0,I))
    return recon.mean(), kl.mean()


def step(x):
```

```
    N = len(x)
    h, mu, lv = encode(x)
    eps = rng.standard_normal(mu.shape)
    z = mu + np.exp(0.5 * lv) * eps                  # reparameterization trick
    g, logits = decode(z)
    recon, kl = losses(x, logits, mu, lv)
    # ----- backpropagation -----
    dlog = (sigmoid(logits) - x) / N
    G = {"W4": g.T @ dlog, "b4": dlog.sum(0)}
    dg = (dlog @ P["W4"].T) * (g > 0)
    G["W3"], G["b3"] = z.T @ dg, dg.sum(0)
    dz = dg @ P["W3"].T
    dmu = dz + mu / N
    dlv = dz * eps * 0.5 * np.exp(0.5 * lv) + 0.5 * (np.exp(lv) - 1) / N
    G["Wmu"], G["bmu"] = h.T @ dmu, dmu.sum(0)
    G["Wlv"], G["blv"] = h.T @ dlv, dlv.sum(0)
    dh = (dmu @ P["Wmu"].T + dlv @ P["Wlv"].T) * (h > 0)
    G["W1"], G["b1"] = x.T @ dh, dh.sum(0)
    return recon, kl, G


# ---------- Training with Adam ----------
if __name__ == "__main__":
    m = {k: np.zeros_like(v) for k, v in P.items()}
    v = {k: np.zeros_like(a) for k, a in P.items()}
    t, epochs, batch, lr = 0, 300, 32, 0.002
    print(f"{'Epoch':<8}{'Train loss':<13}{'Recon':<10}{'KL':<9}{'Test loss'}")
    for ep in range(1, epochs + 1):
        idx = rng.permutation(len(Xtr))
        tot = np.zeros(2)
        for s in range(0, len(idx), batch):
            xb = Xtr[idx[s:s + batch]]
            recon, kl, G = step(xb)
            tot += np.array([recon, kl]) * len(xb)
            t += 1
            for k in P:                                  # Adam update
                m[k] = 0.9 * m[k] + 0.1 * G[k]
                v[k] = 0.999 * v[k] + 0.001 * G[k] ** 2
                mh, vh = m[k] / (1 - 0.9**t), v[k] / (1 - 0.999**t)
                P[k] -= lr * mh / (np.sqrt(vh) + 1e-8)
        if ep == 1 or ep % 50 == 0:
            tot /= len(Xtr)
            _, mu, lv = encode(Xte)
            r, kk = losses(Xte, decode(mu)[1], mu, lv)
            print(f"{ep:<8}{tot.sum():<13.2f}{tot[0]:<10.2f}{tot[1]:<9.2f}{r + kk:.2f}")

    # ---------- Evaluation ----------
    _, mu, lv = encode(Xte)
    rec = sigmoid(decode(mu)[1])
    r, kk = losses(Xte, decode(mu)[1], mu, lv)
    mse = np.mean((rec - Xte) ** 2)
    print(f"\nTest reconstruction error (mean squared, per pixel): {mse:.4f}")
    print(f"Test negative ELBO (per image): {r + kk:.2f}")
    print(f"Latent space dimension: {Z}   mean KL per image: {kk:.2f}")

    # ---------- Generate NEW images by sampling z ~ N(0, I) ----------
    z = rng.standard_normal((30, Z))
    new = sigmoid(decode(z)[1])
    print(f"Generated {len(new)} new images from random latent vectors (saved to image).")

    # Figure 1: originals, reconstructions, new samples
    fig, ax = plt.subplots(5, 10, figsize=(8, 4.3))
    for j in range(10):
        ax[0, j].imshow(Xte[j].reshape(8, 8), cmap="gray_r")
        ax[1, j].imshow(rec[j].reshape(8, 8), cmap="gray_r")
        for r in range(3):
            ax[2 + r, j].imshow(new[r * 10 + j].reshape(8, 8), cmap="gray_r")
    for a in ax.ravel():
        a.set_xticks([])
        a.set_yticks([])
    for r, name in enumerate(["Original", "Reconstructed", "New", "New", "New"]):
        ax[r, 0].set_ylabel(name, fontsize=7, rotation=0, ha="right", va="center")
    plt.tight_layout()
    plt.savefig("img/exp8_samples.png", dpi=200)
    plt.close()
```

```
# Figure 2: latent space and generated manifold
fig, ax = plt.subplots(1, 2, figsize=(9, 4))
sc = ax[0].scatter(mu[:, 0], mu[:, 1], c=yte, cmap="tab10", s=10)
ax[0].set(title="Latent space (test images)", xlabel="z1", ylabel="z2")
fig.colorbar(sc, ax=ax[0], ticks=range(10), label="digit")
n, grid = 15, np.linspace(-2.5, 2.5, 15)
canvas = np.zeros((n * 8, n * 8))
for i, zy in enumerate(grid[::-1]):
    for j, zx in enumerate(grid):
        img = sigmoid(decode(np.array([[zx, zy]]))[1]).reshape(8, 8)
        canvas[i * 8:(i + 1) * 8, j * 8:(j + 1) * 8] = img
ax[1].imshow(canvas, cmap="gray_r", extent=[-2.5, 2.5, -2.5, 2.5])
ax[1].set(title="Images generated across the latent space", xlabel="z1", ylabel="z2")
plt.tight_layout()
plt.savefig("img/exp8_latent.png", dpi=200)
```

## OUTPUT:

```
Terminal - Variational autoencoder

root@ip-10-10-5-10:~# python3 exp8.py
Epoch   Train loss   Recon     KL       Test loss
1       38.31        38.02     0.28     29.60
50      24.94        22.79     2.15     23.96
100     24.60        22.25     2.36     23.79
150     24.52        21.97     2.54     23.84
200     24.35        21.74     2.61     23.70
250     24.23        21.54     2.69     23.64
300     24.21        21.41     2.80     23.66

Test reconstruction error (mean squared, per pixel): 0.0359
Test negative ELBO (per image): 23.66
Latent space dimension: 2   mean KL per image: 2.78
Generated 30 new images from random latent vectors (saved to image).
```

**Fig 3: Original, reconstructed and newly generated images**

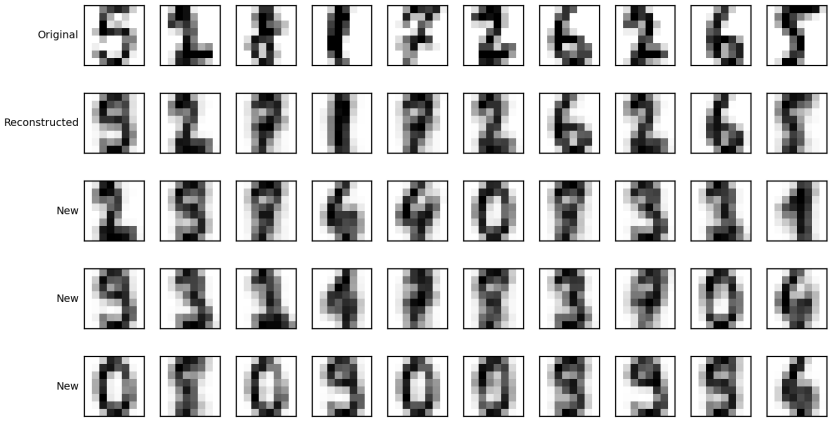
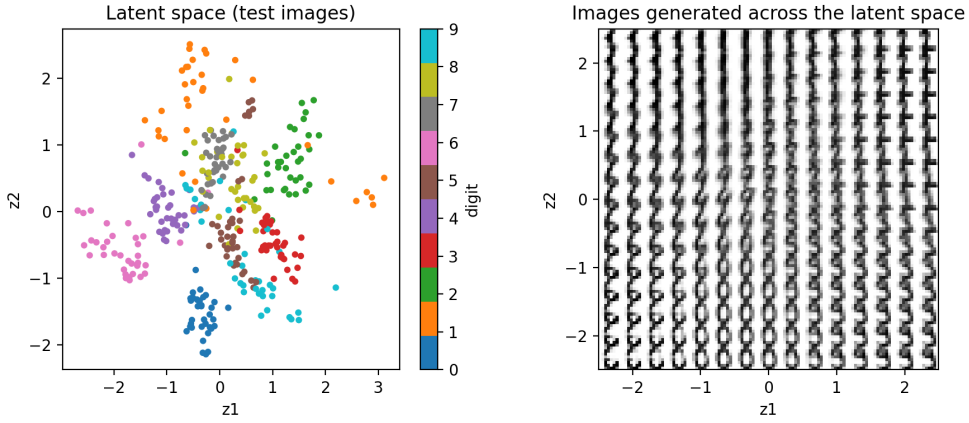



**Fig 4: Learned latent space and images decoded across it**




## RESULT:

Thus a variational autoencoder was implemented from scratch and trained on handwritten digit images. On the test images it reached a negative ELBO of 23.66 per image and a reconstruction error of 0.0359 (mean squared, per pixel). In the 2-dimensional latent space several digits such as 0, 6 and 2 formed clear groups while others overlapped, and sampling z from N(0, I) produced new digit-like images. With only 2 latent dimensions and 8x8 images the generated digits are blurry and some are hard to read. The dataset is the 8x8 digits set, not CelebA.